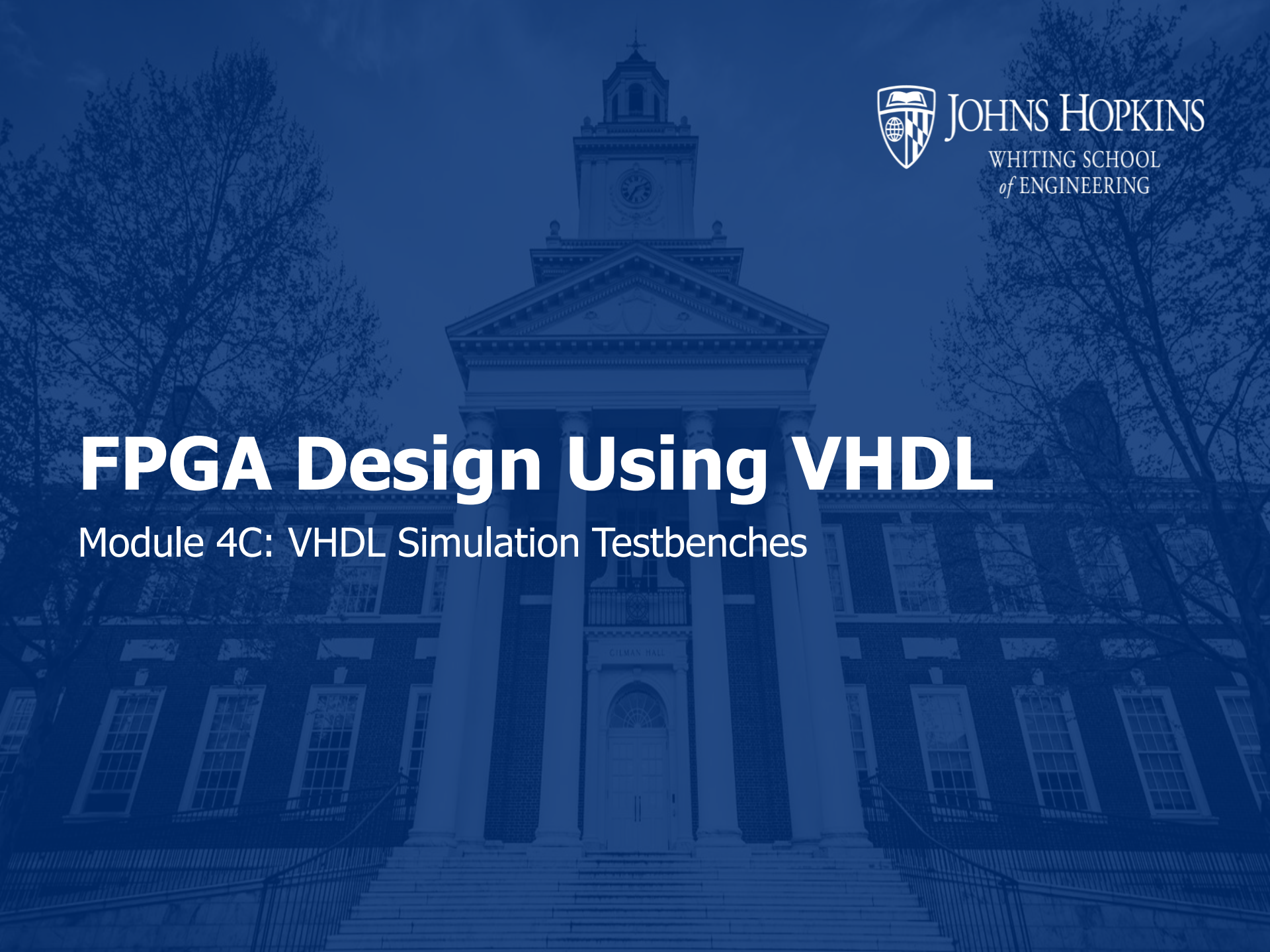
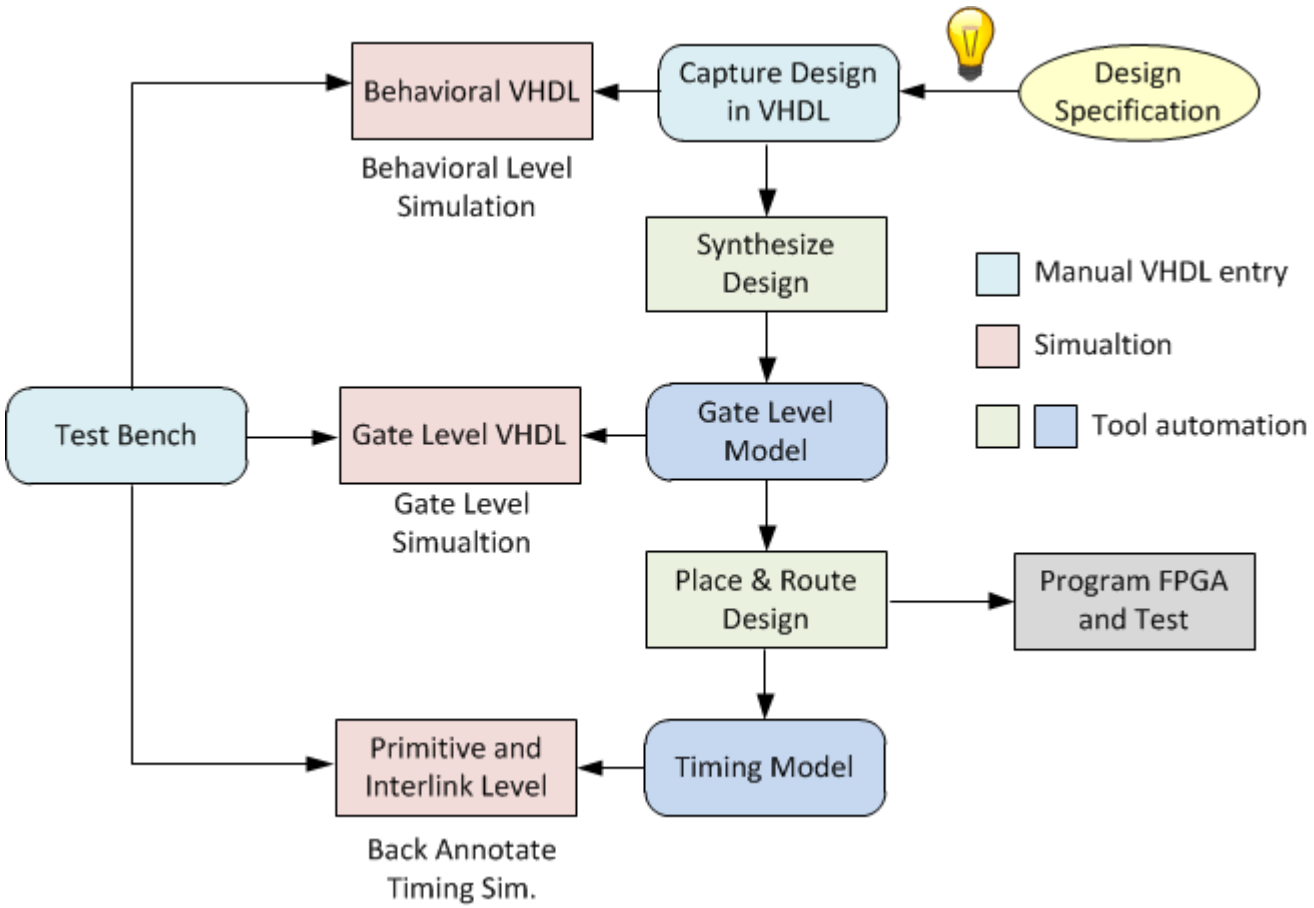


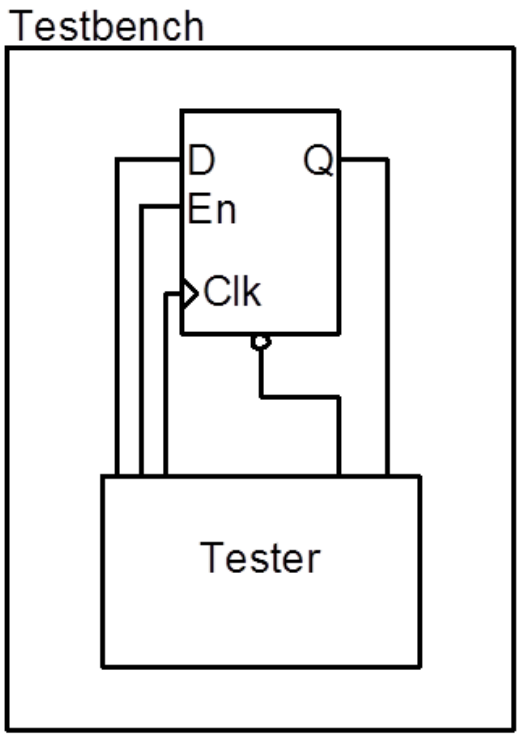
FPGA Design Using VHDL

Module 4C: VHDL Simulation Testbenches

VHDL Testbenches



Simple Testbed Example




```
library ieee;
use ieee.std_logic_1164.all;
use work.all;

entity dff_tb is
end entity dff_tb;

architecture model of dff_tb is

--Signals
signal clk    : std_logic;
signal reset  : std_logic;
signal enable : std_logic;
signal d      : std_logic;
signal q      : std_logic;
begin
    .
    .
    .
end architecture model;
```

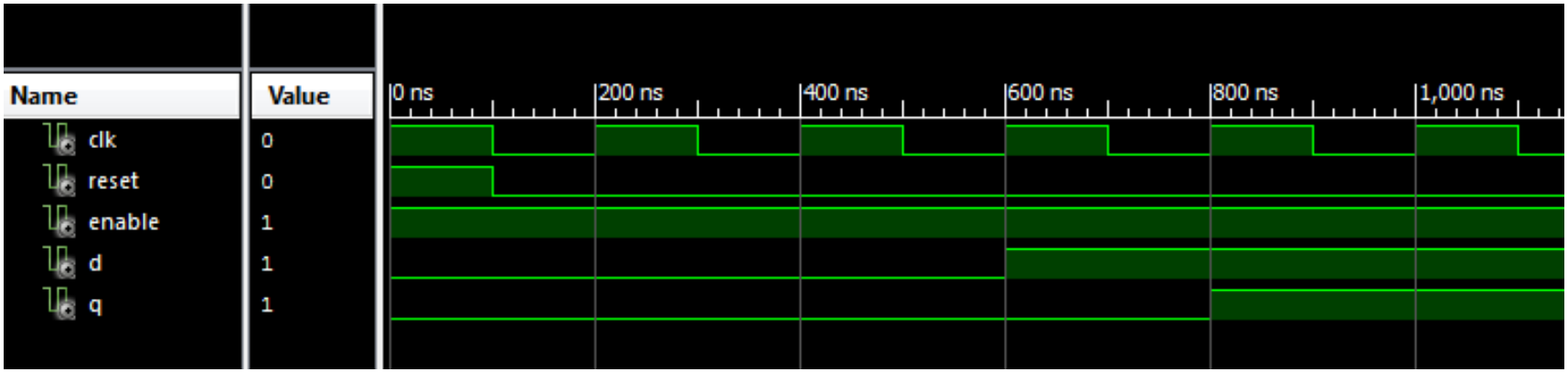
In this simple case "tester" will be a set of process statements in the testbench architecture.

Simple Testbed Example Continued

```
begin
   dut : entity dff port map
   (d => d, clk => clk, reset => reset, enable => enable, q => q);

   genclk: process is
   begin
      clk <= '1';
      wait for 100 ns;
      clk <= '0';
      wait for 100 ns;
   end process;

   reset <= '1', '0' after 100 ns;
   enable <= '1';

   a_test : process is
      begin
      d <= '0';
      wait until rising_edge(clk);
      wait until rising_edge(clk);
      wait until rising_edge(clk);
      d <= '1';
      wait until rising_edge(clk);
      wait for 1 ps;
      --will this give an error?
      assert '1' = q
         report "ERROR, D did not propogate to Q."
         severity error;
      wait;
   end process a_test;

end architecture model;
```

- Instantiate DFF
- Generate Clock
- Create reset pulse and enable
- Set stimulus and monitor DFF output

Simple Testbed Example Continued 2



Testbenches in VHDL

- Test-benches provide stimulus to the DUT
- Designers can verify designs in simulations
  - Viewing waveforms
  - Automated manner using assertions
- Test-benches consist of
  - Processes
  - CSA
  - Component Instantiations
- **Test-benches are NOT intended for synthesis**

For-Loop Syntax in a Process

```
entity countzeros is port (
  a : in std_logic_vector (7 downto 0);
  Count : out std_logic_vector (3 downto 0) );
end countzeros;

architecture behavior of countzeros is
  signal Count_i: unsigned (3 downto 0);
begin
  process (a)
  begin
     Count_i <= "000";
     for i in 7 downto 0 loop
        if (a(i) = '0') then
           Count_i <= Count_i + 1;
        end if;
     end loop;
     Count <= std_logic_vector(Count_i);
  end process;
end behavior;
```

- Not necessary to define `i`
- Complete form:

```
for i in integer range 7 downto 0 loop
```

- Note
  - `for-loop` in this manner is not intended for synthesis

Exit Statement

```
entity count_trail is Port (
   vec: in std_logic_vector(15 downto 0);
   count : out unsigned (4 downto 0));
end count_trail;

architecture Behavioral of count_trail is
begin
   process(vec)
      -- notice short-hand method for initializing variable for simulation
      variable result : unsigned(4 downto 0) := "00000";
   begin
      for i in 15 downto 0 loop
         exit when vec(i) = '1';
         result := result + 1;
      end loop;
      count <= result;
   end process;
end Behavioral;
```

While-Loop Syntax in a Process

```
process(vec)
   variable result : unsigned(4 downto 0);
   variable i : integer;
begin
   result := "00000";
   i := 0;
   while i < 16 loop
      exit when vec(i) = '1';
      result := result + 1;
      i := i + 1;
   end loop;
   count <= result;
end process;
```

- Must have defined range at compile time
- Can't have infinite loops

`Wait` Statement

- **`wait`** used in simulation to specify when to suspend execution of process and when to resume
- Examples:
  - **`wait for`** `20 ns;  -- suspends the process, and waits for 20 ns`
  - **`wait on`** `clk, reset;  -- suspends process until an event on clk` **`or`** `reset`
  - **`wait until`** `enable = '1';`
- Used mostly in **testbenches** for **simulation**,
  - **Non-synthesizable (some are, some are not)**

`Wait` Statement – Synthesizable?

- Non-synthesizable

```
process
begin
   if(rising_edge(clk)) then
      wait for 20 ns;
      Q <= D;
   end if;
end process
```

- `Wait for` statement unsupported

- Synthesizable

```
process
begin
   wait until clk = '1';
   pulsedelay <= pulse;
end process;
```

Note that a process with a `wait` statement in it SHOULD NOT have a sensitivity list
A process with a sensitivity list CANNOT have a `wait` statement in it
"`statement WAIT not allowed in a process with a sensitivity list`"

`Wait` Statement Continued

- Infinite loop process

```
gen_clk : process
begin
    clk <= '1';
    wait for 100 ns;
    clk <= '0';
    wait for 100 ns;
end process gen_clk;
```

- Process executed once

```
set_resetn : process
begin
    resetn <= '0';
    wait until rising_edge(clk);
    wait for 50 ns;
    resetn <= '1';
    wait;
end process set_resetn;
```

Note that a process with a `wait` statement in it SHOULD NOT have a sensitivity list
A process with a sensitivity list CANNOT have a `wait` statement in it
"`statement WAIT not allowed in a process with a sensitivity list`"

Variables

```
process(A, B, Ci)
    variable var_s1,
             var_s2,
             var_s3 :
             std_logic;
begin
    S <= A XOR B XOR Ci;
    var_s1 := A AND B;
    var_s2 := B AND Ci;
    var_s3 := A and Ci;

    Co <= var_s1 OR
var_s2 OR var_s3;
end process;
```

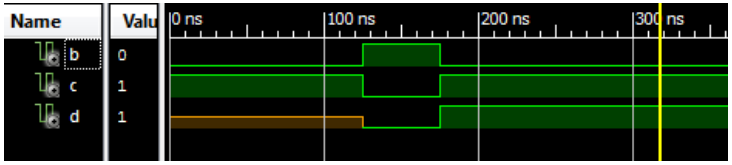
- Variables are for storing (an) intermediate computation(s) in the middle of a process
- Good for organizational purposes

Simulation Statement

- Statements in a process are executed in order in a simulator
- Each statement is executed every **delta cycle**
  - All statements are executed in zero **simulation time**
- All **signals** are **updated** at the **end** of the process
  - In simulation
- All variables are updated **immediately**
  - In simulation

Variables in Simulation

```
-- Process without variables
example : process(b)
begin
    c <= not b;
    d <= not c;
    c <= b;
    c <= not b;
end process example;
```




```
-- Process with variables
example2 : process(b)
   variable c_var : std_logic;
   variable d_var : std_logic;
begin
   c_var := not b;
   d_var := not c_var;
   c_var := b;
   c_var := not b;
   c <= c_var;
   d <= d_var;
 end process example2;
```